

```

367 #
368 #
369 #
370 #
371
372 def picks(data, row, n=1):
373     """Mutate n random x-columns."""
374     s = row[:]
375     for col in sample(data.cols.xs,
376                       min(n, len(data.cols.xs))):
377         s[col.at] = pick(col, s[col.at])
378     return s
379
380 def oneplus1(data, mutate, accept, oracle,
381             budget=1000, restart=0):
382     """(1+1) search: mutate, score, accept."""
383     h, best, best_e = 0, None, 1E32
384     s, e, imp = choice(data.rows)[:], 1E32, 0
385     while h < budget:
386         for sn in mutate(s):
387             h += 1
388             en = oracle(sn)
389             if accept(e, en, h, budget):
390                 s, e = sn, en
391                 if en < best_e:
392                     best, best_e, imp = sn[:], en, h
393                     yield h, best_e, best
394             if restart and h - imp > restart:
395                 s = choice(data.rows[:])
396                 e, imp = 1E32, h
397             break
398
399 def oracleNearest(data, row):
400     """Score: copy y-vals from nearest."""
401     near = nearest(data, row)
402     for col in data.cols.ys:
403         row[col.at] = near[col.at]
404     return disty(data, row)
405
406 #
407 #
408 #
409 #
410
411 def same(xs, ys, eps):
412     """Are two lists statistically same?"""
413     xs, ys = sorted(xs), sorted(ys)
414     n, m = len(xs), len(ys)
415     if abs(xs[n//2]-ys[m//2]) <= eps: return True
416     gt = sum(bisect.bisect_left(ys, a) for a in xs)
417     lt = sum(m-bisect.bisect_right(ys, a) for a in xs)
418     if abs(gt-lt)/(n*m) > the.stats.cliffs:
419         return False
420     ks = lambda v: abs(bisect.bisect_right(xs, v)/n
421                      -bisect.bisect_right(ys, v)/m)
422     return max(max(map(ks, xs)), max(map(ks, ys))
423               ) <= the.stats.conf * ((n+m)/(n*m))**.5
424
425 def bestRanks(d):
426     """Group treatments tied for best."""
427     items = sorted(d.items(), key=lambda kv:
428                   sorted(kv[1])[len(kv[1])//2])
429     k0, lst0 = items[0]
430     best = {k0: adds(lst0, Num(k0))}
431     for k, lst in items[1:]:
432         if same(lst0, lst, spread(best[k0])*the.stats.eps):
433             best[k] = adds(lst, Num(k))
434         else: break
435     return best
436

```

```

437 #
438 #
439 #
440 #
441
442 def dinc(k1, k2, b4=None):
443     """Increment nested dict counter."""
444     b4=b4 or {}; b4[k1]=b4.get(k1) or {}
445     b4[k1][k2] = b4[k1].get(k2, 0) + 1
446     return b4
447
448 def o(x):
449     """Recursive format. Sorts dicts."""
450     if isa(x, float):
451         return f"{x:.{the.show.decimals}f}"
452     if isa(x, dict):
453         return "{" + ",".join(f"{k}={o(v)}"
454                               for k, v in sorted(x.items())) + "}"
455     if isa(x, list):
456         return "[" + ",".join(map(o, x)) + "]"
457     if isa(x, S): return "S" + o(x.__dict__)
458     if hasattr(x, "__dict__"):
459         return x.__class__.__name__ + o(x.__dict__)
460     return str(x)
461
462 def table(lst, w=10):
463     """Print list of dicts as aligned table."""
464     if not lst: return
465     ds=[x if type(x) is dict else x.__dict__ for x in lst]
466     ks = list(ds[0].keys())
467     print("".join(f"{str(k):>{w}}" for k in ks))
468     print("-" * (len(ks) * w))
469     for d in ds:
470         print("".join(
471             f"{str(d.get(k,'')):>{w}}" for k in ks))
472
473 def thing(txt):
474     """Coerce string to number or bool."""
475     txt = txt.strip()
476     b=lambda s: {"true":1, "false":0}.get(s.lower(), s)
477     for f in [int, float, b]:
478         try: return f(txt)
479         except ValueError: pass
480
481 def nest(t, k, v):
482     """Set value in nested namespace."""
483     for x in (ks := k.split("."))[:-1]:
484         t = t.__dict__.setdefault(x, S())
485     setattr(t, ks[-1], v)
486
487 def csv(f, clean=lambda txt:
488        txt.partition("#")[0].split(",")):
489     """Yield typed rows from a CSV file."""
490     with open(f, encoding="utf-8") as file:
491         for txt in file:
492             row = clean(txt)
493             if any(x.strip() for x in row):
494                 yield [thing(x) for x in row]
495
496 def pick(it, v=None):
497     """Sample from distribution."""
498     if Sym == type(it): return pick(it.has)
499     if Num == type(it):
500         tmp=v if v is not None and v!="?" else it.mu
501         lo,hi=it.mu-3*it.sd,it.mu+3*it.sd
502         new=tmp+it.sd*2*(rand()+rand()+rand()-1.5)
503         return lo + (new-lo) % (hi-lo + 1e-32)
504     if dict == type(it):
505         n = sum(it.values()) * rand()
506         for k, v in it.items():
507             if (n := n - v) <= 0: break
508         return k

```

```

509 #
510 #
511 #
512 #
513
514 the = S()
515 for k,v in re.findall(r"([w.]+)=([S+]*, __doc__):
516     nest(the, k, thing(v))

```